EEC 172 Final Project

Ziyi Chen
University of California, Davis
Davis, United States
czych@ucdavis.edu

Ayden Leu
University of California, Davis
Davis, United States
ajleu@ucdavis.edu

ACM Reference Format:
Ziyi Chen and Ayden Leu. 2026. EEC 172 Final Project. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Description

This project implements a cloud-connected remote drawing system using the CC3200 microcontroller, an IR receiver, and an OLED display. After powering on, the CC3200 connects to Wi-Fi using its onboard wireless module and initializes the drawing interface on the OLED screen. A user interacts with the system using a standard TV remote control. When a button is pressed, the IR receiver detects the infrared signal and converts it into digital high-low pulses that are decoded by the CC3200. Based on the decoded input, the microcontroller updates the drawing canvas and displays the result on the OLED screen, allowing the user to move a cursor and create drawings in real time. When the user chooses to save the drawing, the CC3200 uploads the image to the cloud. The device first requests a pre-signed URL from an AWS API Gateway endpoint connected to a Lambda function. This function generates a secure upload link for an Amazon S3 bucket. Using the returned URL, the CC3200 sends a PUT request to upload the image. Once the image is stored, S3 triggers another Lambda function that automatically sends the saved image as an email attachment using Amazon SES.

2 Design

2.1 System Architecture

See **Figure 1**.

2.2 Functional Specification

See **Figure 2**.

3 Implementation

3.1 System Architecture

3.1.1 Connecting to Wi-Fi: Note: This specific section is re-using template code from one of our previous labs.

(1) Set the SimpleLink to its default state.
(2) Start the SimpleLink device.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

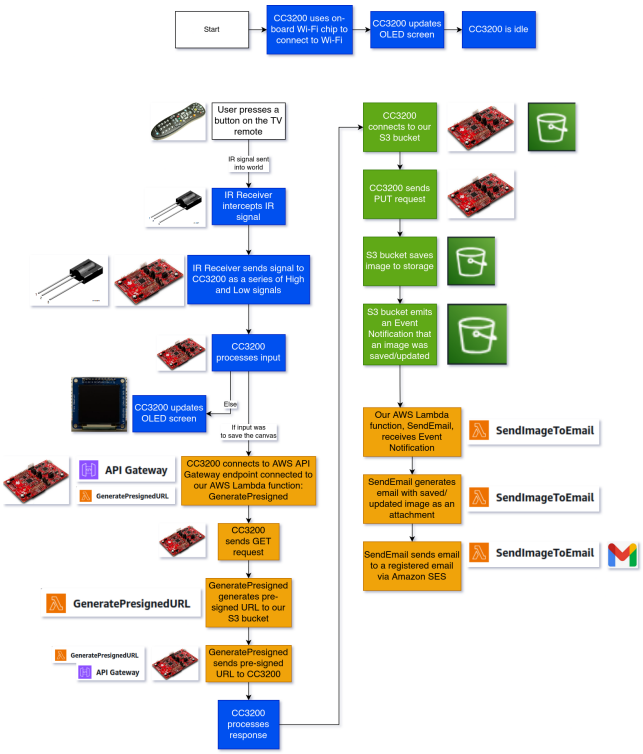

Figure 1: System Architecture

(3) Connect to the a preset wlan network. For us, that would be a phone hotspot network named "F" with no password and configured to use a 2.4 GHz band.
(4) Update the device to the current time and date so encryption can be used.

3.1.2 Updating the OLED screen: Note: This specific section is re-using code from one of our previous labs.

(1) Create a pin configuration according to **Table 1**.
(2) Wire up the CC3200 LaunchPad and its peripherals according to **Table 2**.
(3) For each pixel color value in the `uint16_t*` canvas buffer:
  (a) Get the corresponding screen position from this pixel's canvas position.
  (b) Update the corresponding pixel on the OLED screen.
(4) Draw pixels that represent the cursor at the cursor's screen position.

3.1.3 Processing IR Receiver Data:

(1) During initialization:

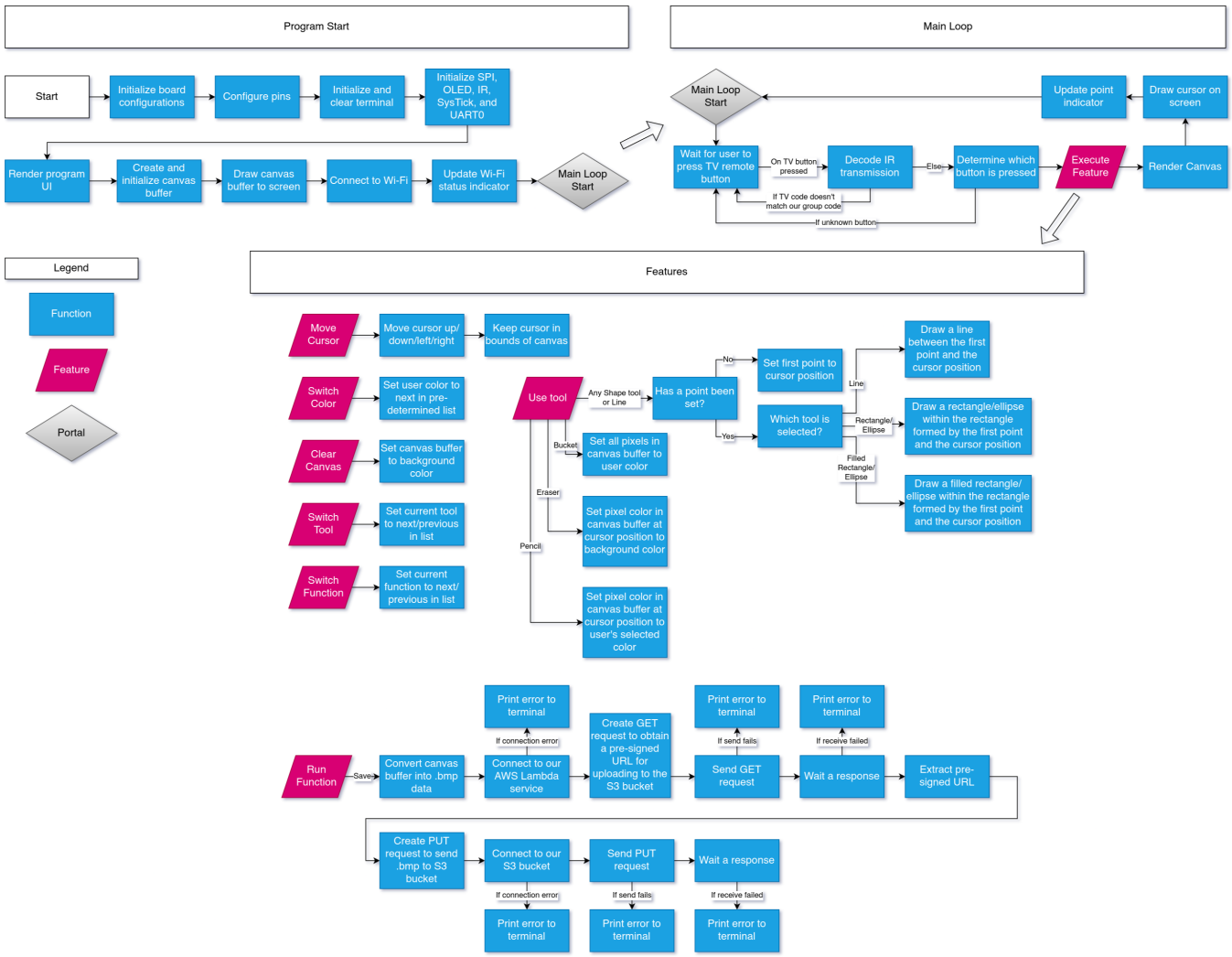

Figure 2: Functional Specification

Field	Pin	In/Out Type
GPIO06	61	Output
GPIO09	64	Output
GPIO11	2	Output
GPIO22	15	Output
GPIO28	18	Output
GPIO31	45	Output
SPI CLK	5	N/A
SPI MOSI	7	N/A
UART0 RX	57	N/A
UART0 TX	55	N/A

Table 1: Pin Configuration for CC3200 LaunchPad

(a) Link an interrupt handler function that will run whenever the pin connected to the IR receiver's output (`P2.2`) receives a rising edge.
(b) Setup the SysTick module to keep track of time.
(c) Create a buffer that will store SysTick time values for each rising edge.
(d) Create a counter to keep track of the number of rising edges captured.
(e) Create a boolean that will tell the program to start decoding the SysTick time values in the buffer.

(2) When the interrupt handler function runs:
(a) Determine if the IR receiver caused the interrupt by checking the interrupt status against the IR receiver's pin.
(b) Store the current SysTick time value for later.
(c) Get the amount of time between the current SysTick time value and the SysTick time value of the previous interrupt call.
(d) Convert the time difference to μ seconds for easier comparison.
(e) Store the time difference into a buffer for later use.

Pin From	Pin To
OLED SI	CC3200 P2.6
OLED CL	CC3200 P1.7
OLED DC	CC3200 P1.5
OLED R	CC3200 P2.4
OLED OC	CC3200 P2.10
OLED +	CC3200 VCC
OLED G	CC3200 GND
IR Receiver GND	GND
IR Receiver Vs	100μ F Capacitor + 100Ω Resistor
100μF Capacitor	GND
100Ω Resistor	CC3200 VCC
IR Receiver OUT	CC3200 P2.2

Table 2: Wire Configuration for CC3200 LaunchPad and Peripherals

(f) Increment the counter that keeps track of the amount of rising edges captured.
(g) If the counter is equal to or above the number of rising edges we're expecting, set the previously created boolean to true.

(3) In `main()`, loop forever and check the the value of the boolean.
(4) If the boolean is true:
  (a) Create a copy of this data in case another IR receiver signal is received.
  (b) Let the program receive more IR receiver signals while processing is happening.
  (c) Find the first time difference that is greater than 8000 microseconds.
    (i) From our analysis of the TV remote signals and testing, a duration of 8000 microseconds usually indicated the start of an IR receiver signal being sent.
  (d) Process each time difference in the buffer.
    (i) If it's greater than 1700 microseconds, it corresponds to a 1 bit.
    (ii) Otherwise, it corresponds to a `0` bit.
  (e) If the number of processed time differences is less than 32, then some data was lost during the interrupt process. The current decoded bits are then discarded. Otherwise, the full payload was received and we have fully decoded it.

3.1.4 *Connecting to an AWS API Gateway/S3 Bucket Endpoint:*

(1) Determine which server and port you want to connect to.
  (a) For our AWS API Gateway, it was, without spaces, <id> . execute-api . <region> . amazonaws.com
  (b) For our S3 Bucket, it was, without spaces, <bucket-name> . s3 . <region> . amazonaws.com
  (c) Our port was just 443, which is the standard HTTP port.
(2) Create a TLS connection with that server on that port.

3.1.5 *Creating and sending a GET request, and Awaiting a Response:*

(1) Determine which host you want to send your GET request to.
  (a) For us, this ended up being, without spaces, <id> . execute-api . <region> . amazonaws.com
(2) Create a TLS connection with said host on port 443, the standard HTTP port.
(3) Determine the endpoint you want to send your GET request to.
  (a) For us, this ended up being, without spaces, / default / <name-of-lambda-function>
(4) Assemble a string payload in the following format. Each entry is a new line that must be set, or else it may not work. The values we put will be included.
  (a) GET /default/<name-of-lambda-function> HTTP/1.1\r\n
  (b) Host: <id>.execute-api.<region>.amazonaws.com\r\n
(5) Send your payload to the host using `sl_Send()`.
  (a) Parameter 1 is the returned value after connecting to the host.
  (b) Parameter 2 is the paylod.
  (c) Parameter 3 is the length of the payload.
  (d) Parameter 4 is `0`, as no flags have to be set.
(6) Wait for a response using `sl_Recv()`.
  (a) Parameter 1 is the returned value after connecting to the host.
  (b) Parameter 2 is the place to store the received value. It will usually be a string.
  (c) Parameter 3 is the size of place to store the received value.
  (d) Parameter 4 is `0`, as no flags have to be set.

3.1.6 *Lambda Function Pre-signed URL Generation for Uploading to an S3 Bucket:* And how to send the URL back to whoever ran the function.

(1) Setup an AWS Lambda function and attach a policy with the following settings:
  (a) Action: `s3:PutObject`
  (b) Resource: `arn:aws:s3:::<bucket-name>/*`
(2) In the code for the Lambda function, do the following:
  (a) Import:
    (i) "S3Client" and "PutObjectCommand" from "@aws-sdk/client-s3"
    (ii) "getSignedUrl" from "@aws-sdk/s3-request-presigner"
  (b) Create a S3Client and set its region to the region your S3 bucket belongs to.
  (c) Export an asynchronous anonymous function that does the following points.
  (d) Create a new `PutObjectCommand` with:
    (i) "Bucket" set to the name of your S3 bucket.
    (ii) "Key" set to the name of the file you want to modify/save into the S3 bucket. Include the file extension.
    (iii) "ContentType" set to the type of the file. For us, it was `image/bmp`.
  (e) Generate new signed URL with "getSignedURL()" and await its return value. This is why the asynchronous anonymous function was created.
  (f) Return an object with the following fields:
    (i) "statusCode: 200": This means nothing went wrong during the process.
    (ii) "body: <json-object>": The data to send back. <json-object> has the following fields.
      (A) "uploadURL: url": "url" is return value of "getSignedURL()"

(B) "key: key": The second "key" is the name of the file previously mentioned.

3.1.7 Processing the Response: The received value will be a giant JSON string represented as a `char` array. There are many ways to process a JSON string, but our method just involved finding the start and end indices of the wanted URL's endpoint URL and saving that to a new variable.

3.1.8 Creating and sending a PUT request, and Awaiting a Response:

(1) Determine what data you want to send in the PUT request.
  (a) For us, it was our canvas buffer converted into the `.bmp` format.
(2) Determine which host you want to send your PUT request to.
  (a) For us, this ended up being, without spaces, <s3-bucket-id> . s3 . <region> . amazonaws.com
(3) Create a TLS connection with said host on port `443`, the standard HTTP port.
(4) Determine the endpoint you want to send your GET request to.
  (a) For us, this was the pre-signed URL endpoint we processed in a previous step.
(5) Assemble a string payload in the following format. Each entry is a new line that must be set, or else it may not work. The values we put will be included.
  (a) PUT <pre-signed-url-endpoint> HTTP/1.1\r\n
  (b) Host: <s3-bucket-id>.s3.<region>.amazonaws.com\r\n
  (c) Content-Type: image/bmp\r\n
  (d) Content-Length: <size-of-data>
(6) Send your payload to the host using `sl_Send()`.
  (a) Parameter 1 is the returned value after connecting to the host.
  (b) Parameter 2 is the payload.
  (c) Parameter 3 is the length of the payload.
  (d) Parameter 4 is `0`, as no flags have to be set.
(7) Send your data immediately after using the same command, with payload replaced with your data. If your data is too big like it was for us, the program will just hang forever, so consider sending it in chunks.
(8) Wait for a response using `sl_Recv()`.
  (a) Parameter 1 is the returned value after connecting to the host.
  (b) Parameter 2 is the place to store the received value. It will usually be a string.
  (c) Parameter 3 is the size of place to store the received value.
  (d) Parameter 4 is `0`, as no flags have to be set.

3.1.9 Making S3 Bucket Save to Storage: Sending the PUT request to the pre-signed URL does this automatically, so no configuration is needed to achieve this affect.

3.1.10 Making S3 Bucket Emit an Event:

(1) Go to your S3 bucket's properties.
(2) Under "Event notificcations," create an event notification with the following settings:
  (a) Event types: Put. This will make the S3 buccket emit an event whenever a file gets added/updated.
  (b) Destination: Lambda function. This is the Lambda function that will receive the event information and do things.

3.1.11 Lambda Function Sending an Email with a File from S3 Bucket Storage:

(1) Under "Amazon SES" > Configuration > Identities, create an identity for:
  (a) The email that will be seen as being "from" when the subscriber looks at the email.
  (b) The subscribers' email.
(2) Setup an AWS Lambda function and in its code, do the following:
  (a) Import:
    (i) "S3Client" and "GetObjectCommand" from "@aws-sdk/client-s3".
    (ii) "SESClient" and "SendRawEmailCommand" from "@aws-sdk/client-ses".
  (b) Create an instance of `S3Client` and set its region to the region where the S3 bucket is located.
  (c) Create an instance of `SESClient` and set its region to the same region.
  (d) Export an asynchronous Lambda handler function that processes the incoming event.
  (e) Extract the S3 event information from the Lambda trigger:
    (i) Retrieve the first record from `event.Records`.
    (ii) Extract the S3 bucket name from `record.s3.bucket.name`.
    (iii) Extract the file name from `record.s3.object.key` and handle any spaces or URL-encoded characters.
  (f) Download the file from the S3 bucket by creating a new `GetObjectCommand` with the following parameters and awaiting its result:
    (i) "Bucket" set to the extracted S3 bucket name.
    (ii) "Key" set to the file name.
  (g) Convert the returned S3 object stream into a usable buffer:
    (i) Create an empty array to store data chunks.
    (ii) Iterate through the object's body stream using `for await`.
    (iii) Push each chunk into the array.
    (iv) Concatenate all chunks using `Buffer.concat()` to produce a complete binary buffer.
  (h) Convert the image buffer into a Base64 encoded string using `buffer.toString("base64")` so that it can be attached to an email.
  (i) Define a multipart boundary string for the next step. It cana be as simple as "NextPart."
  (j) Construct a raw MIME email message in the following format:
    (i) From: <email that the subscriber will see its from>
    (ii) To: <subscriber email>
    (iii) Subject: <the email subject line>
    (iv) MIME-Version: 1.0
    (v) Content-Type: multipart/mixed; boundary="<the boundary string>"
    (vi) –<boundary string>
    (vii) Content-Type: text/plain; charset="UTF-8"
    (viii) <the text body of the email>

(ix) –<boundary string>
(x) Content-Type: image/bmp; name="<file name>"
(xi) Content-Disposition: attachment; filename="<file name>.<file extension>"
(xii) Content-Transfer-Encoding: base64
(xiii) <the Base64 encoded string>
(xiv) –<boundary string>

(k) Create a new `SendRawEmailCommand` using the constructed raw email message, converting the message into a buffer.
(l) Send the email by calling `ses.send(command)` and awaiting its completion.
(m) Return an object containing:
  (i) `status: "Email with image sent"` indicating the process completed successfully.

3.2 Drawing Implementation

3.2.1 *Framebuffer Initialization.*

(1) The drawing system uses a framebuffer stored in the `Canvas` structure.
(2) Each element in the buffer represents one pixel in RGB565 format.
(3) The function `Drawing_Init()` allocates memory for the framebuffer.
(4) `Drawing_Reset()` clears the canvas and places the cursor at the center.

3.2.2 *Canvas Rendering.*

(1) The framebuffer stores the current drawing state before it is displayed.
(2) `update_canvas_visual()` iterates through all pixels in the buffer.
(3) Each pixel is drawn to the OLED using `drawPixel()`.
(4) The cursor is rendered on top of the framebuffer by `Drawing_Render()`.

3.2.3 *Move the Cursor.*

(1) Cursor coordinates are stored in `cursor->x` and `cursor->y`. Directional input updates the cursor position through Drawing_MoveCursor().
(2) Boundary checks ensure the cursor remains inside the canvas area.
(3) The cursor is visually displayed as a cross marker on the screen.

3.2.4 *Draw a Pixel.*

(1) The pencil tool draws a pixel at the current cursor position.
(2) The function `draw_pixel()` writes the selected color to the framebuffer.
(3) The pixel index is calculated using `xy_to_buffer_index()`.
(4) The display updates when the framebuffer is rendered to the OLED.

3.2.5 *Erase Pixel.*

(1) The eraser tool replaces the pixel at the cursor location with the background color.
(2) This operation uses the same function as drawing but writes `COLOR_WHITE`.
(3) The framebuffer is updated accordingly.
(4) The updated canvas is then rendered to the display.

3.2.6 *Clear Canvas.*

(1) The function `Drawing_ClearCanvas()` clears the entire framebuffer.
(2) Each pixel is reset to the background color.
(3) This removes all existing drawing content.
(4) The canvas is then refreshed on the OLED display.

3.2.7 *Color Selection.*

(1) The system supports multiple drawing colors stored in a predefined array.
(2) `Drawing_NextColor()` cycles through available colors.
(3) The selected color is stored in `cursor->color`.
(4) A small indicator on the display shows the current color.

3.2.8 *Tool Selection.*

(1) The active tool is stored in `cursor->tool`.
(2) `Drawing_SwitchTool()` cycles through available tools.
(3) Supported tools include pencil, eraser, line, rectangle, ellipse, and bucket.
(4) Tool selection determines how input actions are interpreted.

3.2.9 *Line Drawing.*

(1) Line drawing uses two selected points.
(2) The first activation stores the starting point.
(3) The second activation captures the ending point.
(4) `draw_line()` connects the points using a Bresenham line algorithm.

3.2.10 *Rectangle.*

(1) Two diagonal points define the rectangle.
(2) The boundaries are calculated from the minimum and maximum x and y values.
(3) `draw_rect()` draws the four edges of the rectangle.
(4) `draw_rect_filled()` fills the rectangle using horizontal lines.

3.2.11 *Ellipse / Circle.*

(1) Two points define the bounding region of the ellipse.
(2) `draw_ellipse()` plots symmetric pixels to form the shape.
(3) The algorithm follows a Bresenham-style ellipse rasterization method.
(4) `draw_ellipse_filled()` fills the interior using horizontal lines.

3.2.12 *Bucket Fill.*

(1) The bucket tool fills a connected region starting from the cursor location.
(2) The base color is read from the selected pixel.
(3) `fill_same_color()` recursively replaces adjacent pixels of the same color.
(4) This produces a flood-fill effect similar to paint software.

3.3 UI Interface

Table 3: Remote Control Button Mapping

Button	Function
Vol Up / Vol Down	Change drawing tool
Ch Up / Ch Down	Change operation or tool mode
Mute	Change drawing color
Last	Confirm selection or apply operation
1	Previous animation frame
2	Move cursor up
3	Next animation frame
4	Move cursor left
5	Apply action (draw pixel or set point)
6	Move cursor right
7	/
8	Move cursor down
9	/
0	Clear canvas

4 Challenges

One issue we encountered during the drawing implementation was that after implementing the line drawing feature, we initially reused the same two-point logic for the rectangle tool. However, because the program interpreted the two selected points as line endpoints, the system would always draw a line rather than a rectangle. After analyzing the problem, we realized that the rectangle tool should interpret the two points as the shape's diagonal corners. We correctly determined the rectangle boundaries and drew its four edges by computing the minimum and maximum x and y coordinates from the two selected points.

Another issue occurred when implementing the filled shape and bucket fill functionality. When the cursor was first placed inside a shape and the apply button was pressed, the program only drew a single pixel rather than filling the entire region. To address this issue, we developed a recursive flood-fill algorithm that examines neighboring pixels and replaces all connected pixels with the same base color. This method enables the system to properly fill shapes and regions on the canvas, resulting in behavior similar to the paint bucket tool in common drawing applications.

Yet another issue was setting up everything with Amazon AWS. The main issues stemmed from mis-configured settings in the various services we used for this project. After reading and watching a bunch of tutoritals though, we were eventually able to set everything up correctly.

5 Future Work

Several additional features could be implemented to improve the functionality and usability of the system. One planned feature was a **gyro-based drawing tool**. Using the onboard gyroscope, a virtual ball could move across the canvas as the board is tilted, painting the screen dynamically and creating a more interactive drawing experience.

Another potential improvement is a **polygon drawing tool**, which would allow the user to place multiple points on the canvas. The system would then connect these points with lines and optionally fill the resulting polygon. This would extend the current drawing capabilities beyond simple geometric shapes.

Additional features include **text input support** with a visual pop-up showing the characters being entered, allowing users to annotate drawings directly on the canvas. The system could also support **switching between multiple Wi-Fi networks** and implementing **user authentication** to provide better security for cloud uploads.

Finally, the drawing system could be expanded to support **multiple animation frames**. Users would be able to create a sequence of frames and play them back as an animation, significantly extending the creative capabilities of the device.

6 Bill of Materials

Table 4: Bill of Materials

Component	Purpose
CC3200 Microcontroller	Core processing unit and system control
SPI Interface	Communication with OLED display
AWS IoT Service	Cloud communication and data upload
IR Receiver	Receive commands from IR remote
OLED Display	Display drawing canvas and UI
AT&T S10-S3 Remote	User input device for drawing control